

OutSystems

Professional Knowledge Repository

Technical Handbook · Interview Preparation Guide · Project Portfolio

Gangasagare Ramesh

OutSystems Developer & Associate Consultant

Banking · Energy · Telecom Domains

Digitide | Mumbai, India | June 2026

Table of Contents

Table of Contents	2
Part 1: Project Experience	6
Project 1: MVS Run Phase — Billing System (Energy Domain).....	6
Project MVS — Deep Explanation	6
Project 2: Telecom Customer Onboarding Web App	7
Project 3: Digital KYC Utility for Banking — Web/Mobile	8
Structured Project Explanation	8
Part 2: OutSystems Platform — Structured Notes	10
What is OutSystems?.....	10
Key Features	10
Capabilities of OutSystems.....	10
Core Capabilities.....	10
OutSystems Components & Tools	10
Service Studio Application Layers.....	11
Service Studio Workflow	11
Modular Development	11
Types of Modules	11
Entity & Database Mapping.....	12
Basic Data Types	12
Entity Actions (CRUD)	12
Static Entities	13
Example — Status values:.....	13
Default Attributes.....	13
Server Actions vs Service Actions	13
Server Actions: Your Internal Workhorse.....	13
Service Actions: Your Integration Hero	14
Client vs Server Actions	14
Aggregates.....	15
Capabilities	15
Referential Integrity — Delete Rules.....	15
Database Indexes in OutSystems	15
What is an Index?	15
Key Rules.....	15
Creating Indexes through SQL	16

Get vs GetForUpdate.....	16
OutSystems Application Object (AO).....	16
Role-Based Security.....	16
Built-in Roles.....	16
Custom Roles.....	17
Built-in Server Actions	17
Blocks in OutSystems.....	17
Features.....	17
Part 3: Architecture & Best Practices	18
OutSystems Platform Best Practices Overview	18
1. Naming Conventions	18
2. Reusability	18
3. Database Best Practices.....	18
4. Aggregates vs Advanced SQL.....	18
5. Building for Change	19
6. Architecture Best Practices.....	19
7. Teamwork & Deployment	20
8. Logic & Development	20
9. Security Best Practices.....	20
10. Performance Best Practices.....	21
Circular Reference, Side Reference, Tight vs Loose Coupling	22
What is Circular Reference?	22
What is Side Reference?.....	22
Correct Architecture Flow	22
Tight Coupling.....	23
Loose Coupling	23
OutSystems 20 Best Practices Every Developer Should Know.....	23
1. Design Applications with Modular Architecture	24
2. Use Entity Wrappers Instead of Public CRUD.....	24
3. Use Site Properties Only for Global Configuration	24
4. Avoid Complex Session Variables	24
5. Separate Large Text and Binary Data	25
6. Avoid Aggregate.Count When Possible	25
7. Protect Against SQL Injection	25
8. Create Database Indexes Strategically	25
9. Fetch Only Required Attributes.....	25
10. Avoid Bulk Operations Inside Loops	25

11. Optimize Screen Preparations.....	26
12. Leverage Server Action Caching	26
13. Avoid Isolated Aggregates Returning Full Records.....	26
14. Compress Images.....	26
15. Avoid Large Inline JavaScript and CSS	27
16. Validate Permissions on the Server	27
17. Prevent HTML and JavaScript Injection	27
18. Refactor Large Screens	27
19. Minimize Screen Parameters.....	27
20. Maintain Clean Module Standards	27
Part 4: Deployment — LifeTime vs Service Center.....	28
Simple Way to Remember	28
Deployment Best Practices (from Interview Q&A).....	28
How to do deployment in OutSystems?.....	28
Deployment Best Practices.....	28
How to Create Team and Provide Access.....	29
LifeTime vs Service Center — Detailed Comparison	29
Part 5: Interview Questions & Answers.....	30
Process and Timers.....	30
CSS Interview Questions.....	32
JavaScript Interview Questions	34
Integrations Interview Questions	36
Best Practices & Scenarios.....	36
AI Mentor Studio & Tools	38
Monitoring Tools Reference	38
Part 6: Mobile Best Practices & Additional Reference	39
Mobile Data Layer Best Practices	39
Mobile App Best Practices.....	39
Data Archiving & Purging.....	39
Data Archiving	39
Data Purging	39
Reactive Web Security Best Practices	39
AI Mentor Studio	39
Discovery Tool	40
Quick Reference — Action Flow Rules	40
Quick Reference — Forms & Validations	40
Built-in Validations	40

Custom Validations.....	40
Quick Reference — Relationships	40
Quick Reference — Screens in OutSystems	40
Screen Variables	40
Screen Actions	41
Links & Buttons — Navigation Destinations.....	41

Part 1: Project Experience

This section documents all enterprise-level projects undertaken across Banking, Energy, and Telecom domains — reflecting real-world OutSystems implementation experience.

Project 1: MVS Run Phase — Billing System (Energy Domain)

Worked on E.... MVS, a dutch large-scale energy billing platform managing the complete customer lifecycle, including customer onboarding, meter readings, invoice generation, billing administration, payment reminders, and transaction processing. Developed and enhanced OutSystems modules to improve application usability, performance, and backend efficiency. Designed and optimized Timers (Batch Jobs) for bulk email notifications, payment reminders, billing cycles, invoice generation, and automated transaction processing workflows.

Integrated the iDEAL payment gateway to enable secure online payments and developed REST API integrations for seamless communication with external systems, including request/response mapping and error handling. Built and enhanced End User modules using multiple OutSystems Forge components and developed responsive Reactive Web applications following OutSystems best practices.

Contributed to Oracle PL/SQL (19c) database integration through OutSystems Extension modules and adhered to the OutSystems 3-layer architecture (UI, Core, Foundation) to ensure scalable and maintainable solutions.

Actively participated across DEV, UAT, and PROD environments, collaborating with Agile teams to analyze requirements, deliver enhancements, support deployments, and ensure reliable production operations.

Project MVS — Deep Explanation

E.... is a major Dutch energy company. MVS likely stands for Marketing Verkoop System or a similar internal name — essentially a large-scale utility billing platform built on OutSystems.

What the Platform Does

It manages the entire lifecycle of an energy customer, from the moment they sign up until they pay their bills. Think of it as the backbone system that handles everything between E.... and its thousands of customers.

Core Modules Explained

Module	Description
Customer Onboarding	Registering new customers into the system; capturing contract details, address, meter info, tariff plans; form-based UI + backend logic to validate and store data
Meter Readings	Collecting electricity/gas meter readings (manually submitted or via smart meters); validating readings against

Module	Description
	previous values; these readings feed directly into invoice calculations
Invoice Generation	Calculating consumption based on meter readings × tariff rates; generating itemized invoices (energy used, taxes, fixed charges); producing PDFs or digital invoices sent to customers
Billing Administration	Managing billing cycles (monthly, quarterly, annual); handling adjustments, credits, corrections; tracking billing history per customer
Payment Reminders	Automated follow-ups for unpaid invoices; escalating reminders (1st reminder → 2nd reminder → final notice); triggered by Timers based on due dates
Transaction Processing	Recording payments received; reconciling payments against open invoices; handling partial payments, overpayments, refunds

Technical Contributions

- **Timers (Batch Jobs)** — OutSystems Timers are scheduled background processes. Optimized timers that ran in bulk — e.g., every night, send payment reminders to all customers with invoices overdue by 7 days. Critical for performance at scale — bulk email dispatch, automated status updates, backend cleanup jobs.
- **iDEAL Payment Integration** — iDEAL is the dominant online payment method in the Netherlands (like UPI in India). Integrated the iDEAL API so customers could pay invoices securely online. Involved handling payment initiation, redirect flows, webhook callbacks, and confirming transaction status back into the billing system.
- **REST API Integrations** — Connected the OutSystems platform to external systems — could be third-party meter data providers, postal/email services, tax systems, or bank APIs. Designed consume/expose REST services within OutSystems.
- **Oracle PL/SQL 19c (Extension Modules)** — Some complex data operations (bulk inserts, stored procedures, data migrations) were too heavy for OutSystems logic alone. Wrote/extended Oracle PL/SQL procedures wrapped inside OutSystems Extension modules (using Integration Studio). This is an advanced skill — bridging OutSystems with raw database logic.

3-Layer Architecture

UI Layer → Core Layer → Foundation Layer. Agile environment with sprint-based development with regular releases. Touched all phases: Dev → Testing → Deployment → Production Support (fixing live issues).

Project 2: Telecom Customer Onboarding Web App

Worked on a Portuguese telecom web application, a leading independent FTTH (Fiber-to-the-Home) network provider, supporting broadband service operations through customer data management, service requests, and internal workflow handling.

Developed customer onboarding journeys and service issue handling modules using OutSystems (Service Studio and Integration Studio). Integrated REST APIs for seamless system communication, managed applications through Service Center, and followed a 3-layer architecture to ensure scalable and structured application development.

Project 3: Digital KYC Utility for Banking — Web/Mobile

Structured Project Explanation

I worked on a large-scale digital customer onboarding platform in the banking domain where multiple onboarding journeys were supported, including Salary Account, Current Account, Savings Account, Credit Card linked to Salary Account, and Credit Card add-on journeys. These onboarding flows were available across both mobile and web applications and initially relied primarily on biometric eKYC for customer verification. However, as per RBI regulatory guidelines, banks are required to provide customers with alternative KYC options before enforcing biometric authentication.

Specifically, customers must be allowed to reuse existing KYC data either through Unified KYC (UKYC) — the bank's internal KYC repository — or through CKYC, the Central KYC Registry managed by CERSAI (Government of India). If valid KYC data is successfully fetched from UKYC or CKYC, the onboarding journey should proceed without requiring biometric verification. If UKYC fails to fetch data, returns mismatched data, or encounters API failure, the system must automatically move to CKYC. Additionally, both the customer and the Relationship Manager (RM) are provided with the flexibility to select the preferred KYC method. If CKYC also fails or consent is declined, the process proceeds with physical biometric eKYC as the final fallback mechanism.

KYC Decision Ladder

UKYC → CKYC → Biometric eKYC (final fallback)

To address this requirement, I designed and developed a separate OutSystems Reactive Web (PWA) application called the UKYC–CKYC Utility. This utility acts as a centralized, reusable KYC orchestration module that can be integrated as a dependency into multiple onboarding applications. Instead of embedding KYC logic separately into each onboarding journey, a breakpoint was introduced at the KYC stage. At this point, the main onboarding flow redirects to the Utility for verification handling.

In the mobile onboarding application (used for Savings Account, Current Account, Credit Card journeys, etc.), the redirection is handled within the same mobile application using a Mobile-to-Web in-app browser (web-view) plugin. The Utility executes the complete KYC decision ladder (UKYC → CKYC → Biometric). After successful KYC completion, the user is seamlessly redirected back to the native onboarding application to continue the journey.

The Utility functions as an independent wrapper responsible for handling UKYC and CKYC integrations, consent validation, encryption handling, fallback routing, and secure data mapping. Being built as a PWA in OutSystems

Reactive Web, it is device-agnostic and reusable across mobile, tablet, and web applications, ensuring centralized compliance control, scalability, and reduced duplication of KYC logic across onboarding platforms.

As part of the project, AWS S3 was used extensively for securely storing customer-related data such as OVD document images, KYC master and channel master files, and API logs, providing scalable and durable cloud storage. Additionally, OutSystems Integration Studio was used to integrate with external AWS-hosted databases, enabling secure communication with cloud data sources and ensuring seamless data retrieval, document storage, and audit logging across the onboarding journeys.

 Architecture Highlight

The UKYC–CKYC Utility is built as a PWA (Progressive Web App) making it device-agnostic and reusable across mobile, tablet, and web — a clean, centralized compliance control pattern.

Part 2: OutSystems Platform — Structured Notes

Comprehensive platform concepts, tools, and developer reference notes.

What is OutSystems?

OutSystems is a low-code development platform used to build and deliver enterprise web and mobile applications quickly.

Key Features

- Low-code development environment
- Full-stack development support
- Integration with external systems and databases
- Built-in management and analytics

Capabilities of OutSystems

Core Capabilities

- Integrates applications, code, and databases
- Visual development using: User Interface, Processes, Logic, Data Entities
- Optimized and compiled code generation
- Continuous delivery and deployment support
- Cloud and On-Premise infrastructure support

OutSystems Components & Tools

Tool	Purpose
Platform Server	Handles deployment, compilation, monitoring, and management of applications
Service Studio	Visual development tool for creating web and mobile applications; connects to platform servers
Integration Studio	Creates extensions, integrates external code, works with C# extensions and databases
Service Center	Administrative console for managing servers, monitoring applications, viewing logs and analytics, environment configurations
LifeTime	Application lifecycle management, deployment across environments, managing application versions

Tool	Purpose
Forge	Repository of reusable components that speeds up development

Service Studio Application Layers

Layer	Components	Used For
Process Layer (BPT)	Human tasks, Automated tasks, Decisions, Events, Emails, Timers	Business Process Technology; scheduled actions (Daily, Weekly, Specific intervals)
Interface Layer	UI Flows, Themes, Images, Scripts, Resources	Responsible for UI development
Logic Layer	Client Actions, Server Actions, Integrations, Roles, Exceptions	Defines application business logic
Data Layer	Entity Diagrams, Database tables, Structures, Static Entities, Resources	Data modeling and storage

Service Studio Workflow



Publishing Pipeline

Service Studio → Publish → OutSystems Platform Server → Browser. Publishing compiles and deploys the application automatically.

Modular Development

Types of Modules

Module Type	Used For
App Modules	Creating Reactive Web Apps and Mobile Apps
Blank Modules	Empty starter modules
Service Modules	Service-oriented architecture and shared business logic
Library Modules	Reusable components
Extension Modules	Integrating external databases and custom C# code

**Producers & Consumers**

Producer Module shares elements with other modules. Consumer Module uses shared elements from producers. Helps separate business capabilities across applications.

Entity & Database Mapping

OutSystems Entity	Database Equivalent
Entity	Table
Attribute	Column
Id Attribute	Primary Key
Reference Attribute	Foreign Key
Index	Index
Record	Row

Basic Data Types

Type	Example
Alphanumeric	Text, Email
Numeric	Integer, Decimal
Date & Time	DateTime
Logic	Boolean
Binary Data	File/Image
Referential	Entity Identifier

**Insight**

OutSystems automatically suggests appropriate data types.

Entity Actions (CRUD)

- Create
- Retrieve
- Update
- Delete

**Auto-generated CRUD**

Entity actions are auto-generated by OutSystems.

Static Entities

Special entities used for fixed values like enums.

Example — Status values:

- Active
- Inactive

Default Attributes

- Id
- Label
- Order
- IsActive

Server Actions vs Service Actions

**The Core Difference**

Server Actions are internal workers. Service Actions are bridges to the outside world.

If you've been building in OutSystems for any length of time, you've probably worked with both Server Actions and Service Actions. At first glance, they seem very similar — both run on the server and both return data. But the real question is: Which one should you use, and when?

Server Action	Service Action
Synchronous execution	Asynchronous execution
Same process / transaction	Separate process / network call
Tightly coupled	Loosely coupled
Best for internal logic	Best for external integration
Higher performance	Promotes reusability

Server Actions: Your Internal Workhorse

A Server Action runs within the same process as the logic that calls it — just like a function or method in traditional programming. There's no network hop and no additional overhead. It's the right choice when your logic lives within the same module or tightly related modules.

✓ Synchronous = Predictable

If your Server Action fails, the calling transaction rolls back as well. This is exactly what you want for critical business logic.

Use Server Actions when you need:

- Complex business calculations (loan interest, tax computation, pricing rules)
- Data manipulation within the same application or tightly coupled modules
- Reusable logic shared across screens in the same module (e.g., validation, formatting)

Service Actions: Your Integration Hero

Service Actions are designed for cross-application communication and external integrations. They run in a separate process and involve network communication, making them inherently loosely coupled.

✓ Loosely Coupled = Independently Deployable

When you expose functionality through Service Actions, the consuming application doesn't need to be republished whenever the provider changes.

Use Service Actions when you need to:

- Integrate with payment gateways (Stripe, Razorpay, etc.)
- Connect to external CRM or ERP systems
- Expose shared business logic to multiple OutSystems applications
- Handle any scenario requiring communication across application boundaries

💡 The Golden Rule

If the logic stays inside your application, use a Server Action. If it crosses an application or system boundary, use a Service Action. Making this decision correctly from the beginning can save you from painful refactoring later.

Client vs Server Actions

Client Actions	Server Actions
Execute on device	Execute on server
Faster UI operations	Database operations
Can call client actions	Can call server actions
💡 Insight Client actions improve UI responsiveness.	

Aggregates

Aggregate is a visual query builder used to fetch data.

Capabilities

- Sources
- Filters
- Sorting
- Test Values

Insight

Used instead of writing SQL for simple queries.

Aggregate.Count Warning

Aggregate.Count generates an additional database query. For existence checks, use List.Empty. For counting, create a dedicated lightweight query and remove unnecessary joins and sorting.

Referential Integrity — Delete Rules

Rule	Meaning
Protect	Prevent deletion
Delete	Cascade delete
Ignore	No validation

Database Indexes in OutSystems

What is an Index?

A database index is used to enforce uniqueness of table attributes across multiple rows, improve search performance, and make data retrieval faster using filters. In OutSystems, indexes can be created in the Entity element inside the data model.

Important

Indexes improve read/search performance but adding too many indexes can negatively impact Insert operations, Update operations, and Write performance.

Key Rules

- Avoid creating indexes directly in the database because OutSystems only manages indexes created by the platform.

- If you manually create indexes in the database, you must recreate them during migrations or major Entity changes.
- When a foreign key is created in Service Studio with Delete Rule set to Protect or Delete, OutSystems automatically creates an index with prefix OSIDX.
- If Delete Rule is changed to Ignore, the platform removes the auto-created index.
- OutSystems manages all indexes starting with OSIDX_.
- Do not modify or create indexes with OSIDX_ prefix manually, as custom changes may be lost and can affect application behavior.

✓ Best Practice

Preferred Method: Create indexes directly in Service Studio. Benefits: Managed automatically by OutSystems, safer during deployments and migrations, better maintainability.

Creating Indexes through SQL

If creating indexes manually through SQL tools like SSMS, always use a custom prefix.

```
CREATE UNIQUE INDEX MYINDEX_<internal name>
ON <Entity>(<attributes>)

CREATE INDEX MYINDEX_<internal name>
ON <Entity>(<attributes>)
```

Get vs GetForUpdate

Get	GetForUpdate
Fetches/reads data without locking the record	Locks the record until transaction is completed
Other users can still update at the same time	Prevents other users from modifying the same record simultaneously
Used for normal display operations	Used for safely updating important data

OutSystems Application Object (AO)

An Application Object (AO) is a unit used by OutSystems to measure the size and complexity of applications. It is mainly used for licensing, capacity planning, and monitoring platform usage as the application portfolio grows.

Role-Based Security

Built-in Roles

- Anonymous

- Registered

Custom Roles

Application-specific authorization roles.

Built-in Server Actions

- CheckRole
- GrantRole
- RevokeRole

Security Rule

Never rely solely on UI controls. Button Visible = False is WRONG. Always validate permissions in actions and business logic. Attackers can bypass UI restrictions. Server-side validation is mandatory.

Blocks in OutSystems

Block is a reusable UI component.

Features

- Can contain widgets
- Can contain other blocks
- Has local variables and parameters
- Supports events and placeholders

Insight

Blocks improve UI reusability and maintainability and also save development time.

Part 3: Architecture & Best Practices

Industry-grade OutSystems architectural patterns, best practices, and anti-patterns drawn from real project experience.

OutSystems Platform Best Practices Overview

1. Naming Conventions

- Use meaningful names
- Use PascalCase
- Use Id suffix for foreign keys
- Example: CustomerId, EmployeeCode, Customer_Show
- Group Screens with a prefix (e.g., Customer_Edit and Customer_Show)
- Prefix actions invoked by Timers with Timer_

✓ Short Insight

Consistent naming improves readability and maintenance.

2. Reusability

- Reuse logic using Server Actions
- Use Blocks for reusable UI
- Use User Functions for formatting
- Use RefreshQuery to rerun a Query

✓ Short Insight

Reusable components reduce duplication and improve scalability.

3. Database Best Practices

- Avoid hundreds of attributes in one entity
- Avoid large text fields (text attributes with length above 2000)
- Set mandatory attributes properly
- Configure Delete Rules carefully
- Check the Delete Rule of Foreign Keys to an Entity
- Add descriptions at least to the Entities

✓ Short Insight

Optimized entity design improves application performance.

4. Aggregates vs Advanced SQL

- Prefer Aggregates — they're optimized and database independent!

- Use SQL only when absolutely necessary
- Use parameters in SQL — use EncodeSQL for Inline Parameters
- Avoid hardcoded values — use Parameters instead of hard-coded values in the SQL
- Indent SQL code consistently
- Use Advanced Queries for bulk operations
- Don't put business logic in the SQL
- Keep in mind that Advanced Queries are database specific

✓ Short Insight

Aggregates are optimized and easier to maintain.

5. Building for Change

- Favor Aggregates over SQL Queries
- Keep in mind that SQL Queries are harder to maintain
- Avoid creating custom HTML/JavaScript
- Split functionality in different Web Screens
- Don't use Extensions to implement business logic

✓ Short Insight

Maintainable architecture simplifies future enhancements.

6. Architecture Best Practices

Layer	Purpose
Process Layer	Business workflows
Core Layer	Business logic
Connector Layer	External integrations

Important Practices

- Avoid circular references between modules
- Clearly define the responsibilities of each module
- Remove unused references and unused or duplicate code
- Encapsulate business logic in Server Actions
- Create user story driven interfaces
- Don't isolate the data model in one module — modules should be functional units instead of architectural units
- Use asynchronous logic when possible

✓ Short Insight

Loose coupling improves scalability and deployment flexibility.

7. Teamwork & Deployment

- Use LifeTime for deployment and application versioning
- Avoid publishing single modules: use LifeTime
- Avoid direct production hotfixes without testing first
- Maintain same Service Studio version across team
- Use a single user for each team member
- Avoid working on the same elements at the same time
- Avoid deploying to Production during business hours
- Create team processes for active log monitoring
- Define what 'done' means and include testing

Short Insight

Controlled deployments reduce production risks.

8. Logic & Development

- Remove unused or duplicate code
- A module with a size larger than 4 MB could indicate poor architecture
- Set at least one default button in your Web Screens
- Avoid changing Site Properties values in your logic
- Be careful about infinite recursion
- Hide warnings only for a good reason
- More than 20 site properties in one module normally means bad design
- Avoid using Extensions solely for Web Service consumption
- Disable auditing of Extensions in Production

Short Insight

Large modules often indicate poor architecture design.

9. Security Best Practices

- Set the Screen's Roles
- Be aware of sensitive data exchange
- Don't send sensitive information in screen parameters
- When applicable, use SSL for sensitive information
- Don't rely on the Screen widgets interface to control permissions
- Validate user's Roles before executing Screen Actions
- Use encrypted passwords in the database
- Use Internal Access Only for UI Flows and Web Services
- Check the Reactive web specific security best practices

Short Insight

Backend validation is mandatory for secure applications.

10. Performance Best Practices

Data Model

- Index your Entities
- Isolate large Text and Binary Data in another entity
- Archive old data in separate Entities
- Make good use of the Delete Rules in Entities

Infrastructure

- Setup database maintenance plans
- Configure Web server memory settings
- Backup database transaction logs often
- Tune database file growth
- Use a dedicated server for Timers
- Prefer 64-bit architectures
- Plan and monitor timers schedule so there are no conflicting schedules

Logic

- Look into Service Center reports
- Avoid long-running Timers and batch jobs
- In Traditional Web, simplify screen Preparations
- Place as little information as possible in Session Variables
- Avoid using isolated Aggregates in Server Actions
- Avoid chained Web Service calls — return as much as possible in a single call

Queries

- Don't perform joins over linked servers
- Minimize the number of fields fetched from the database
- Keep Max Records consistent with your needs
- Iterate over queries once and avoid using indexes ([i])
- Minimize the number of executed queries

UI

- Use multiple Screens instead of creating complex ones
- Minimize Screen Parameters
- Import JavaScript files as resources
- Move JavaScript Blocks that aren't needed at loading time to the bottom
- Use CSS Sprite Images

Short Insight

Performance optimization should start during application design.

Circular Reference, Side Reference, Tight vs Loose Coupling

What is Circular Reference?

A Circular Reference happens when modules depend on each other in a loop.

```
Module A → Module B → Module A
Module A uses Module B
Module B again uses Module A
(This creates a circular dependency)
```

Why is it Bad?

- Dependency issues
- Deployment problems
- Poor architecture
- Hard maintenance
- Compilation issues

What is Side Reference?

Side Reference happens when modules from the same layer depend on each other.

```
UI Module A → UI Module B
Both are UI modules in same layer.
(This is called Side Reference.)
```

Why is it Bad?

- Tight coupling
- Poor scalability
- Hard maintenance
- Confusing architecture

Correct Architecture Flow



Dependency Direction

UI Layer → Core Layer → Foundation Layer. Dependencies should always move downward only.

Layer	Responsibilities
UI Layer	Screens, Blocks, Client Actions, UI Logic
Core Layer	Business Logic, Entities, Server Actions, Processes
Foundation Layer	Libraries/Theme, Extensions/External Integrations, Shared Utilities

 Avoid These Anti-Patterns

Circular References, Side References, Tight Coupling

Tight Coupling

Tight Coupling means modules are highly dependent on each other. If one module changes, the other module is also affected.

Problems

- Difficult maintenance
- Hard to scale
- More dependency issues
- Difficult testing
- Low reusability

Loose Coupling

Loose Coupling means modules have minimal dependency on each other. Each module works independently as much as possible.

Advantages

- Easier maintenance
- Better scalability
- Easier testing
- Better reusability
- Cleaner architecture
- Fewer dependency issues

 OutSystems Best Practice

OutSystems recommends: Loose Coupling, Proper 3-layer architecture, Shared logic in Core/Foundation modules. Avoid Circular References.

OutSystems 20 Best Practices Every Developer Should Know

Understanding the Foundation: Before diving into best practices, it's important to understand how OutSystems Web applications work:

- User interacts with the browser
- Browser sends an HTTP request
- Server processes business logic and data
- Server returns an HTTP response
- Browser renders the UI using HTML, CSS, and JavaScript

1. Design Applications with Modular Architecture

OutSystems promotes modular development.

Producer Modules — Expose reusable elements:

- Entities
- Structures
- Server Actions
- Roles
- Web Blocks
- Themes
- Processes

Consumer Modules

Consume public elements from producer modules through dependency management. This improves reuse and reduces duplication.

2. Use Entity Wrappers Instead of Public CRUD

Problem: When multiple modules directly modify public entities, business rules become scattered and difficult to control.

Solution

- Make entities Read Only or private
- Create Server Actions to wrap Create, Read, Update, and Delete operations

Benefits

Centralized validation, Security enforcement, Standard audit fields, Easier maintenance. Consider impact analysis before changing wrapper logic because all consumers depend on it.

3. Use Site Properties Only for Global Configuration

Many developers misuse Site Properties.

Avoid

Using Site Properties for frequently changing data.

Why?

Each update invalidates caches across consumer modules and front-end servers, causing performance degradation.

Recommended

Use Site Properties only for: Environment URLs, Feature toggles, Global constants. Store dynamic configurations in database entities.

4. Avoid Complex Session Variables

Complex Session Variables create database overhead.

Best Practice

Store only lightweight session information. For complex user state management: Create dedicated session entities and retrieve data on demand.

5. Separate Large Text and Binary Data

Large text and binary fields slow down database operations.

Avoid

- Text attributes > 2000 characters
- Storing images/files in frequently accessed entities

Better Approach

Split into dedicated entities: Profile, ProfileBio, ProfilePicture. This improves query performance and reduces record size.

6. Avoid Aggregate.Count When Possible

Many developers don't realize that Aggregate.Count generates an additional database query.

Better Alternatives

- For existence checks: List.Empty
- For counting: Create a dedicated lightweight query. Remove unnecessary joins and sorting.

7. Protect Against SQL Injection

Dynamic SQL introduces security risks.

```
-- Use EncodeSQL() for user inputs
-- For IN Clauses use:
BuildSafe_InClauseIntegerList()
BuildSafe_InClauseTextList()
-- Never directly concatenate user input into SQL statements
```

8. Create Database Indexes Strategically

Indexes can dramatically improve query performance. Focus on: Frequently filtered fields, Join conditions, Date fields, High-cardinality columns. Avoid excessive indexing because it increases write costs.

9. Fetch Only Required Attributes

Bad Practice: Retrieving entire entity records when only a few fields are needed. Better Practice: Use Structures, Optimized Aggregates, Specific attribute selection. Less data transferred = faster application.

10. Avoid Bulk Operations Inside Loops

Slow Pattern

```
For Each
  UpdateEntity()
End
```

Better Pattern

```
UPDATE Entity
SET Status = 'Active'
WHERE Condition

-- Or:
DELETE FROM Entity
WHERE Condition
```

One SQL statement is significantly faster than thousands of iterations.

11. Optimize Screen Preparations

Slow preparations directly impact user experience.

Avoid

- Queries inside loops
- Complex security checks per screen
- Heavy preprocessing

Prefer

- Pre-calculated data
- Session initialization logic
- Efficient joins

Keep preparation logic lightweight.

12. Leverage Server Action Caching

If a Server Action is repeatedly called — e.g., Country lists, Static configurations, Lookup data — enable caching for the action. Benefits: Reduced database calls, Faster page loads, Improved scalability.

13. Avoid Isolated Aggregates Returning Full Records

When reusable actions return full entity records, platform optimizations are reduced and more data is fetched than necessary. Better: Return Structures containing only required fields.

14. Compress Images

Large images significantly impact first-load performance. Example: A 4.1 MB image reduced to 60 KB loads dramatically faster. Best Practices: Resize before upload, Compress images, Use modern formats when possible.

15. Avoid Large Inline JavaScript and CSS

Problems: Larger page size, No caching, Difficult maintenance. Recommended: Move JavaScript to external files, CSS to centralized themes. This improves performance and maintainability.

16. Validate Permissions on the Server

Critical Security Rule

Never rely solely on UI controls. Wrong: Button Visible = False. Right: Always validate permissions in actions and business logic. Attackers can bypass UI restrictions. Server-side validation is mandatory.

17. Prevent HTML and JavaScript Injection

When rendering dynamic content, use:

```
EncodeHTML()  
EncodeJavascript()  
SanitizeHtml()  
// Avoid writing custom HTML with unescaped user input
```

18. Refactor Large Screens

Large screens cause: Slow rendering, Heavy network payloads, Longer processing times. Solution: Break screens into smaller screens, reusable blocks, lazy-loaded content. Show information only when required.

19. Minimize Screen Parameters

Passing entire records between screens impacts performance. Better: Pass OrderId instead of Order Record. Retrieve the data in the destination screen.

20. Maintain Clean Module Standards

Recommended Practices

- Follow naming conventions — Examples: Customer_CS, Common_UI, Theme_Th
- Move large resources to separate modules
- Add descriptions to public actions, blocks, and parameters

These practices improve discoverability and maintenance.

Key Takeaways — Top 5

1. Wrap entity CRUD operations with Server Actions. 2. Avoid queries inside loops. 3. Use Structures instead of full entities whenever possible. 4. Validate permissions on the server side. 5. Optimize data retrieval and screen performance.

Part 4: Deployment — LifeTime vs Service Center

A clear reference guide to OutSystems deployment tools and best practices.

Simple Way to Remember

Tool	Purpose	Think of It As
LifeTime	Moving applications across environments (Development → QA/Testing → Production)	The deployment pipeline manager
Service Center	Managing everything within a single environment — logs, timers, site properties, manually deploying .oap files	The environment admin console
Service Studio → Publish	First Deployment — always starts here, deploys directly to Development environment	The initial developer publish step

First Deployment Rule

First Deployment = Always starts from Service Studio → Publish, which deploys the application directly to the Development environment. For this initial step, neither LifeTime nor Service Center is required.

Deployment Best Practices (from Interview Q&A)

How to do deployment in OutSystems?

- Use LifeTime to perform application versioning and cross-environment deployment
- Avoid publishing single modules: use LifeTime
- Avoid applying a hotfix without testing it first

Deployment Best Practices

Before deployment:

- Republish application
- Deploy Core → Business → UI
- Inform development team

Impact Warning

If another developer publishes during deployment, the deployment may fail or throw errors. Best Practice: No publishing during deployment.

How to Create Team and Provide Access

Create Team in Lifetime under User Management.

LifeTime vs Service Center — Detailed Comparison

Aspect	LifeTime	Service Center
Scope	Across all environments	Single environment only
Primary Use	Deploy apps between Dev / QA / Prod	Manage runtime settings, logs, timers
Application Versioning	Yes — built-in versioning and tagging	Limited — no cross-env versioning
Monitoring	Aggregated performance, end-user experience	Detailed logs, analytics, integration logs
API Performance Check	End-user experience monitoring	Check Integration Logs in Service Center
Timer Management	No	Yes — run timers manually from Service Center
Site Properties	No	Yes — configure per environment
.oap File Deployment	No	Yes — manual deployment via .oap

Part 5: Interview Questions & Answers

Structured interview preparation covering OutSystems core concepts, CSS, JavaScript, Integrations, DevOps, and Best Practices.

Process and Timers

Q Why do we use timers in our application? What are the purposes of using timer?

Timers are asynchronous logic in OutSystems. It is useful in:

- Running batches
- Loading bulk data in tables
- Doing automated checks for data
- Deleting data
- Sending emails at a predetermined time
- Fetching environment report through email
- Configuring application after deployment

Q What will happen when timer gets an exception?

Once timer gets an exception:

- The DB operations done by the timer will be rollback
- It will retry 2 more times
- Then it gets aborted

We can change timer execution attempts from Service Center.

Q Suppose a timer is scheduled at an interval of 5 mins and it is taking more than 5 mins due to large number of data coming through an API. When will next scheduled timer run?

It will wait for the current timer to finish the execution then it will run.

Q What is BPT and why we use this?

OutSystems allows you to design and manage your business processes and integrate them into your applications.

A business process is simply called a Process in OutSystems and is understood as the way that a particular task is carried out in your organization.

Examples:

- Handling invoices
- Processing orders

- Handling complaints

Processes are also known as BPT (Business Process Technology).

It can be used inside action; it will run in background.

Q What is the difference between BPT and Light BPT?

A Process in OutSystems can be:

Light BPT: Will activate process immediately after data manipulation on an Entity.

Important Difference: Triggers execute within same transaction, whereas Light BPT starts separately.

BPT: Used more for digitalizing workflows.

Q Can we run BPT on click?

Yes, we can run BPT on click through the action.

It can also run inside timer but it needs proper handling for: Timeout and Idle process.

Q Suppose a user uploads a large excel document and validation + insertion causes timeout. How can you solve this issue?

Server/Service action with validation for large amount of data will get timed out.

We can use BPT in this scenario:

- Runs in background
- Validates data
- Inserts data

For very large amount of data:

- Break data into smaller chunks inside BPT
- Prevent timeout issues

Q How will you debug a Timer?

- Add breakpoints inside action
- Start debugging from same module
- Run timer from Service Center

Q What happens if Timer schedule is set to 'When Published'?

It is not a best practice.

Whenever module gets published:

- Timer automatically triggers
- Can impact higher environments

- May clear data or insert bulk records unexpectedly

Q How to handle heavy timer?

If timer takes more than default timeout (20 mins):

- Increase timeout
- Process data in chunks
- Commit frequently

Recommended Batch Size: 1000–2000 records

Q Can we schedule timer in 1 min interval?

Yes, but:

- Need to manually provide schedule
- Max interval approx. 300–310 intervals (~5 hrs)
- To cover 24 hrs: Multiple timers required

CSS Interview Questions

Q What is the order of precedence of CSS style?

Precedence / Loading Order (lowest to highest):

1. Blocks (lowest)
2. Base theme(s)
3. Theme and Theme Editor
4. Screen
5. Styles Editor
6. Inline styles

Q Scenario based CSS precedence question: Theme CSS = RED, Screen CSS = PINK, Web Block CSS = YELLOW. What is the final background color?

Final background color will be: Yellow

Because Web Block CSS takes precedence.

Q What are selectors in CSS?

Selectors are used to find and select HTML elements.

Types of Selectors:

- Simple selectors
- Combinator selectors
- Pseudo-class selectors
- Pseudo-elements selectors
- Attribute selectors

Q What are Pseudo-classes?

Pseudo-class defines special state of an element.

Examples: Mouse hover, Visited links, Focus state

Q How to highlight element on mouse over?

Using pseudo class:

```
div:hover {  
  background-color: Red;  
}
```

Q Change first child's color of an element.

```
p i:first-child {  
  color: blue;  
}
```

Q What are CSS combinators?

Combinator explains relationship between selectors.

Types:

- Descendant selector (space)
- Child selector (>)
- Adjacent sibling selector (+)
- General sibling selector (~)

Q What are Pseudo-Elements?

Pseudo-element styles specific part of an element.

Examples:

- First letter
- First line
- Before content

- After content

Q Change first line color of element.

```
p::first-line {  
  color: #ff0000;  
  font-variant: small-caps;  
}
```

Q Select nth child element.

```
div:nth-child(2) {  
  background: red;  
}
```

Q What is Universal selector?

Universal selector (*) selects all HTML elements.

```
* {  
  text-align: center;  
  color: blue;  
}
```

Q Difference between Flex and Grid?

Grid → Two-dimensional layout

Flex → One-dimensional layout

Q What is position static, relative and absolute?

position: static — Default positioning.

position: relative — Position relative to current position.

position: absolute — Position relative to closest positioned ancestor.

JavaScript Interview Questions

Q In which scenarios we use JS in OutSystems?

OutSystems allows developers to write custom JavaScript code inside applications.

Used for:

- Functionality enhancement
- Interactivity
- Frontend customization

Q How does OutSystems integrate JavaScript into its platform?

OutSystems allows developers to write custom JavaScript code inside applications.

Used for: Functionality enhancement, Interactivity, Frontend customization

Q How to include custom JavaScript code?

JavaScript can be added:

- In JavaScript files
- In Scripts section of screen/web block

Q How to invoke JavaScript functions from OutSystems?

Using:

- RunJavaScript action
- JavaScript expression

Q How to manipulate HTML elements using JavaScript?

Using DOM methods:

- getElementById()
- querySelector()
- Event listeners

Q Limitations while using JavaScript?

Need to consider:

- Security risks
- Cross-browser compatibility
- Performance
- Maintainability

Q What are JavaScript callbacks?

Callbacks are functions passed into other functions and executed later.

Used in:

- Async integrations

- API response handling

Q Considerations for JavaScript in mobile apps?

Need to consider:

- Device compatibility
- Performance optimization
- Platform limitations

Q How to handle cross-browser compatibility?

- Follow best practices
- Use feature detection
- Use standard APIs
- Test across browsers

Integrations Interview Questions

Q What is onBeforeRequest and onRequest?

onRequest: Runs logic before executing exposed REST API request.

onBeforeRequest: Used to modify URL, Headers, Request text. Example: Token-based authentication

Q What is onAuthentication method?

Used to authenticate consumer in OutSystems.

Q How to check API performance?

Check Integration Logs in Service Center.

Q Where to check API exceptions?

Service Center → Integration Logs

Best Practices & Scenarios

Q Timeout issue with 20k records in Data Action. How to resolve?

Use BPT (Business Process Technology) for background processing.

Break data into smaller chunks: Recommended Batch Size: 1000–2000 records.

Process data in chunks and commit frequently.

Increase timeout if necessary.

Q A screen is rendering very slow. How will you investigate?

Check:

- Aggregate refreshes
- Server actions in OnInitialize / OnReady
- Slow SQL warnings
- Data action timing
- Integration logs
- API response time

Q Static Entity Auto Number issue in upper environment?

If Auto Number = Yes:

- IDs generated automatically
- IDs may differ across environments
- Hardcoded IDs may create issues

Q Can we use inline CSS?

Yes, but:

- Hard to maintain
- Not best practice

Q What is Sanitization API?

Used to prevent:

- HTML injection
- JavaScript injection
- SQL injection

Q Why should we not frequently change Site Properties?

Changing Site Properties:

- Restarts IIS
- Impacts performance

AI Mentor Studio & Tools

Q What is AI Mentor Studio and Discovery Tool?

AI Mentor Studio — Used for: Technical debt analysis, Performance analysis, Security analysis, Architecture validation.

Discovery Tool — Used for: Reviewing solution architecture, Finding architecture violations, Dependency analysis

Q Explain debugging and testing in OutSystems.

- Add breakpoints inside action
- Start debugging from same module
- For Timers: Run timer from Service Center
- Check Service Center logs for runtime errors
- Use Integration Logs for API debugging

Monitoring Tools Reference

Tool	Provides
Service Center	Logs, Analytics, Monitoring reports
LifeTime	Aggregated application performance, End-user experience monitoring
Performance Monitoring API	Detailed performance analysis
App Feedback	Users can report issues and suggest improvements
AI Mentor Studio	Technical debt, Performance, Architecture, Maintainability, Security analysis

Part 6: Mobile Best Practices & Additional Reference

Mobile-specific guidelines, data archiving concepts, and reactive web security practices.

Mobile Data Layer Best Practices

- Use Local Storage
- Reduce server calls
- Improve navigation speed
- Support offline mode

Mobile App Best Practices

- Keep heavy logic on server side
- Use Fetch Data actions
- Use On After Fetch
- Group server calls
- Test in real-world scenarios

Data Archiving & Purging

Data Archiving

Process of moving old data into secondary storage.

Benefits

- Better runtime performance
- Cost savings

Data Purging

Permanent deletion of inactive/obsolete records.

Reactive Web Security Best Practices

- Do not expose sensitive client-side data
- Fetch only required fields
- Protect screens with roles
- Validate server-side security

AI Mentor Studio

AI Mentor Studio analyses: Performance, Architecture, Maintainability, Security.
It helps identify: Technical debt, Best practice violations, Optimization opportunities.

Discovery Tool

Used for: Reviewing solution architecture, Identifying architecture violations, Dependency analysis.

Quick Reference — Action Flow Rules

- Starts with one Start node
- Can have multiple End nodes
- Supports conditions using If/Switch
- Uses Assign to set variable values

Quick Reference — Forms & Validations

Built-in Validations

Automatically validates: Mandatory fields, Correct data types.

Custom Validations

Implemented manually in screen actions. Properties: Valid, Validation Message.

Quick Reference — Relationships

Relationship Type	Description	Example
1-to-1	One record linked to one record	Profile ↔ ProfileDetail
1-to-Many	One record linked to multiple records	One Customer → Many Orders
Many-to-Many	Implemented using a junction entity	Orders ↔ Products

Quick Reference — Screens in OutSystems

Screen: A UI building block containing widgets and logic.

Screen Variables

- Input Parameters
- Local Variables

Screen Actions

Client-side logic executed within a screen.

Links & Buttons — Navigation Destinations

- Screen
- External URL
- Client Action
- Event

End of Document

OutSystems Professional Knowledge Repository — Gangasagare Ramesh — June 2026